

CRAFT Research Assistant

Pitch for the Phase 1 prototype · October 2026

This prototype is a working chat assistant. CSA engineers ask questions in plain language and get answers that are drawn only from the approved document collection. Every fact carries a citation that opens the original document at the exact place it came from.

The design follows the CRAFT requirements document (CSA-CRAFT-RD-0001 V3) closely. There is one accountable agent per conversation. That agent is free to plan its own research, but every action it takes passes through defined checkpoints. Its private workspace exists only for the life of the conversation, and its citations are checked by the system rather than taken on trust.

Answers you can check
Each citation names the document, its exact version and the section, page or row. The system rejects any citation it cannot match to a real passage.

Autonomous, but governed
The agent decides how to search, and every tool use passes a checkpoint. That is the control point the requirements call for (§1.3, UMR-012).

Sealed by default
Each conversation and its tools run in their own Docker container. When the conversation ends, the agent and its files are removed, and the step-by-step record of what it did can be kept.

How to read this document

The architecture is described through five **views**. Each one shows the same system from a different angle and answers one question.

View	The question it answers	Where
Context view	Who uses CRAFT, and which outside systems does it depend on?	Figure 1
Container view	What are the main building blocks, and how do they talk to each other?	Figure 2
Component view	What happens inside a single conversation?	Figure 3
Dynamic views	What happens, step by step, when someone asks a question, or when an action needs approval?	Figures 4–5
Deployment view	Where will it run in production?	Figure 6

Throughout the figures, **solid** outlines are working today and **dashed** outlines are planned. Requirement identifiers (for example UMR-006) point to the requirements document.

1. Context view: who and what CRAFT deals with

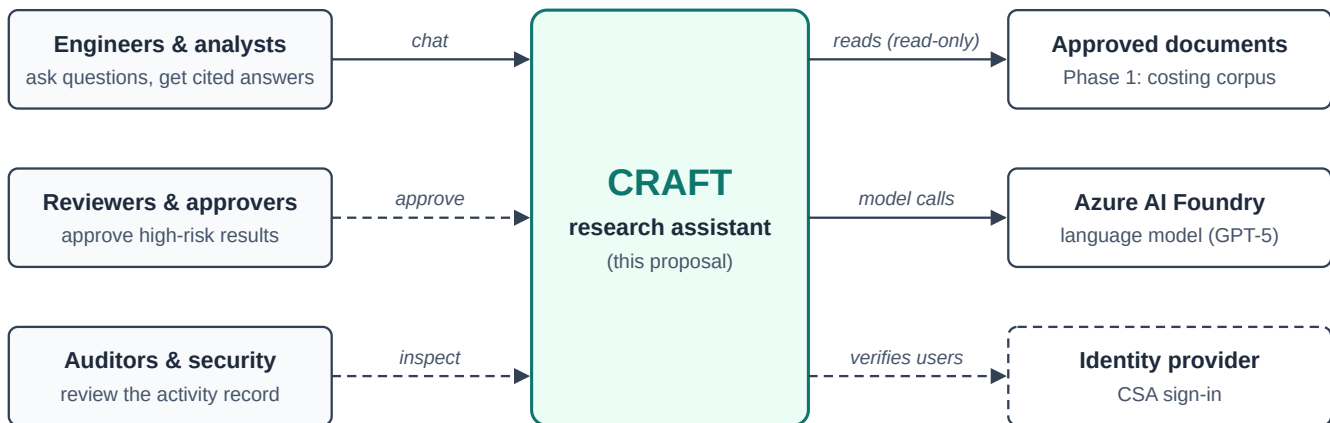


Figure 1 · Context view. CRAFT sits between the people who use it and three outside systems. It only reads the approved collection and never writes to it. Approvals, audit access and CSA sign-in are planned (dashed).

2. Container view: the main building blocks

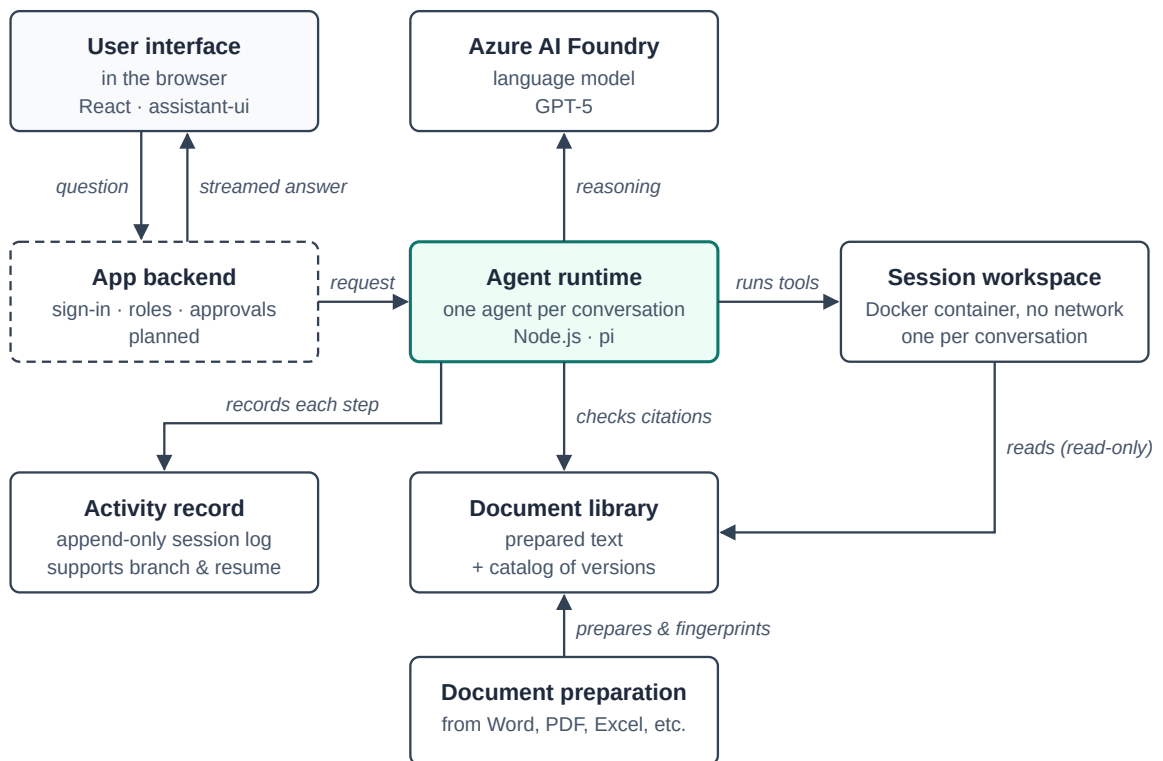


Figure 2 · Container view. The agent runtime (highlighted) is the centre of the system. Today the user interface talks to it directly. The planned app backend will sit in between to handle sign-in, roles and approvals, so the agent runtime never holds user identities. Each conversation's tools run in its own Docker container, which can only read the prepared copies of documents; the original files never reach it. The agent framework writes every step to the activity record, which can also be used to branch or resume a conversation.

3. Component view: inside one conversation

CRAFT's agent works the way a capable researcher does. It plans, searches, reads, and searches again with better terms until it has enough evidence. The requirements ask for exactly this freedom (UMR-094). They also ask that control sit at the point where the agent *uses a tool*, rather than in a fixed script (§1.3, UMR-012). Figure 3 shows that control point.

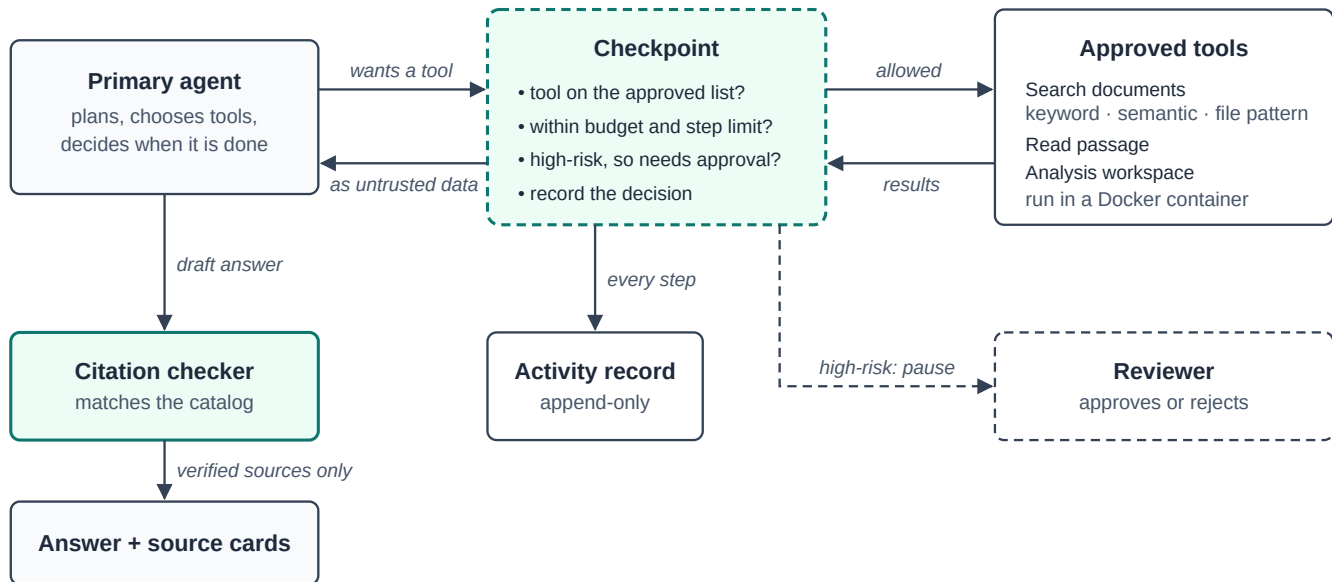


Figure 3 · Component view. Every tool request passes the checkpoint before it runs, and every result passes back through it before the agent sees it. Document text comes back labelled as data, never as instructions (UMR-068). The tools run inside the conversation's own Docker container. The citation checker and the activity record work today: the agent framework (pi) writes every message, tool call and result to an append-only log. The checkpoint rules and the reviewer step are the next build stage. pi already provides a hook at exactly this point for every tool call, so they plug in without changing the agent.

What this design gives CSA

Research that adapts to the question. Looking up a requirement ID, comparing costs across missions and tracing a risk each call for a different search. The agent chooses the approach and keeps going until it has the evidence.

Rules that can grow without rebuilding. New controls, such as an approval step, a spending limit or a newly approved tool, are added at the checkpoint. The agent itself doesn't change, and every step it takes is still recorded.

4. Dynamic views: what happens, step by step

Asking a question

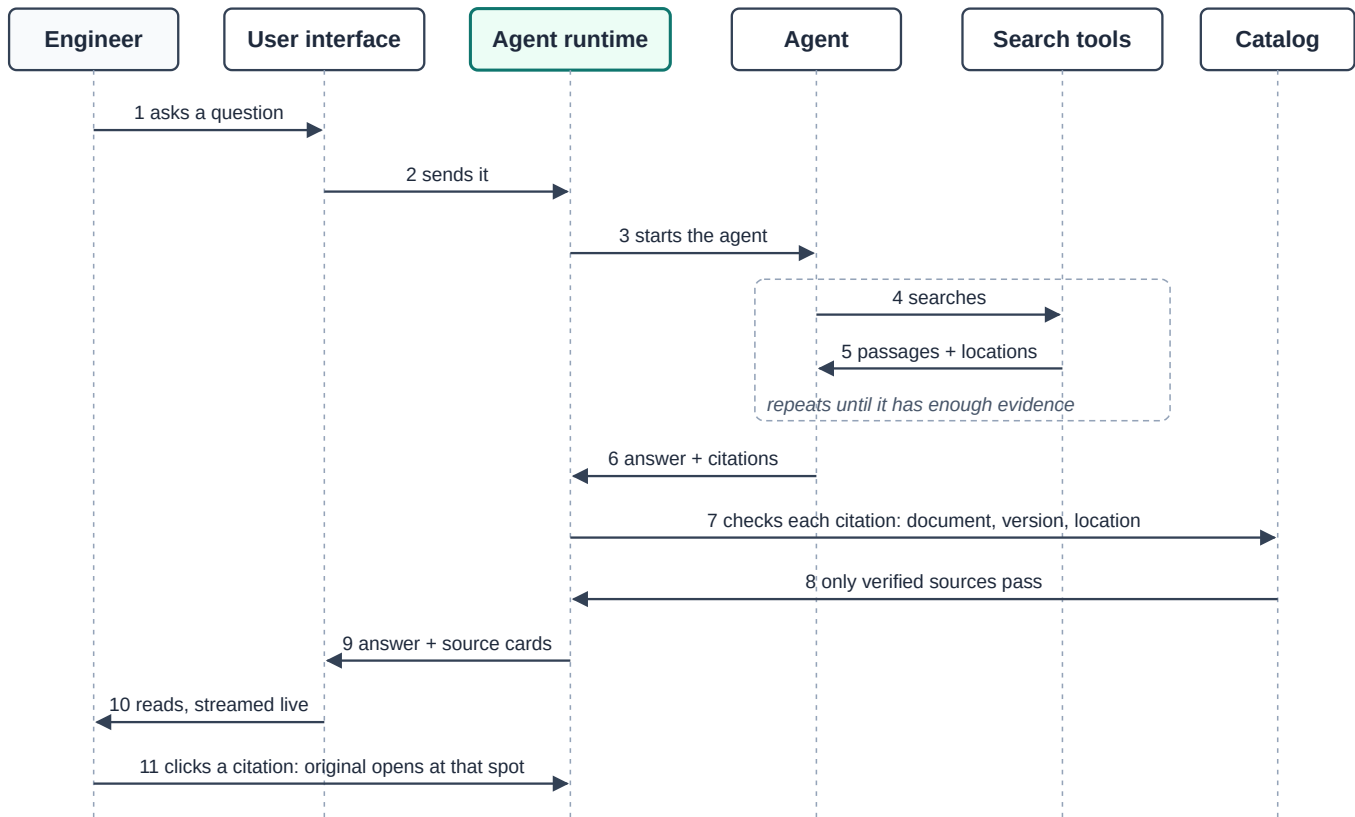


Figure 4 · Asking a question. The agent decides for itself how many searches it needs (steps 4–5). The answer is streamed as it is written. Before any source card is shown, the runtime confirms that the cited document, version and location really exist (steps 7–8). Clicking a citation opens the original Word file at the cited heading, the PDF at the cited page, or the spreadsheet at the cited row.

When an action needs approval

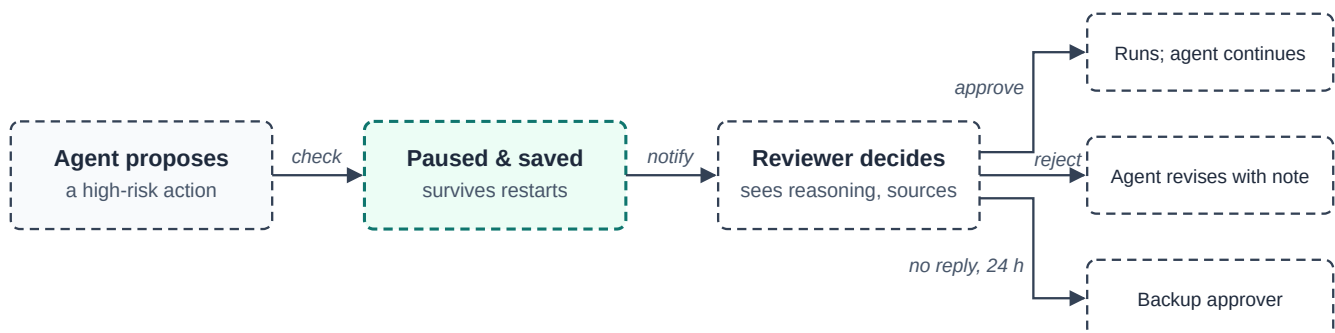


Figure 5 · Approval flow (planned). When the checkpoint sees a high-risk action, for example exporting files out of CRAFT, it pauses the agent and saves the conversation to disk, so a pending approval survives a restart. The reviewer sees the agent's reasoning, its sources and its search path (UMR-021 to UMR-026). The user interface library already includes approve/deny cards.

5. Deployment view: where it runs

The prototype runs on a single Azure virtual machine. The production design keeps the same building blocks and moves each one onto a managed Azure service inside a Protected B landing zone in a Canadian region (UMR-058, UMR-070).

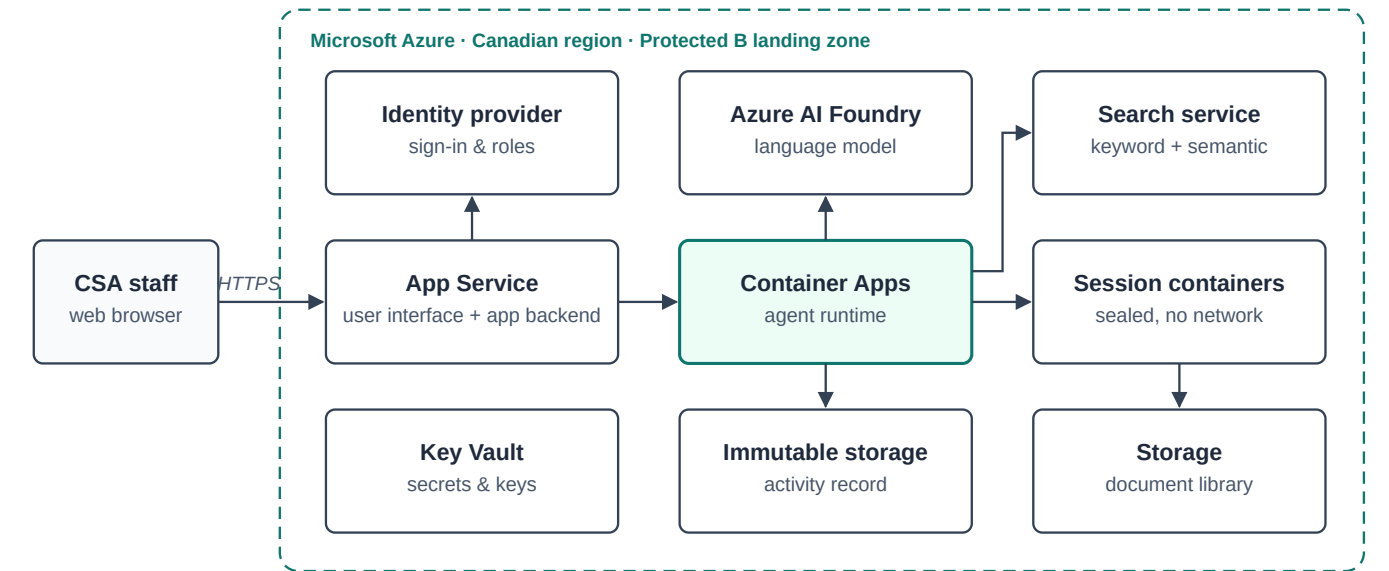


Figure 6 · Deployment view (end state). Each block from Figure 2 has a managed Azure home. The agent runtime calls the model, the search service and the sealed session containers, and writes every step to storage that cannot be altered or deleted. Session containers read the document library read-only and have no network access.

Building block	Prototype today	Production
Language model	Azure AI Foundry (OpenAI GPT-5)	Azure AI Foundry in a Canadian region. A configuration change, not a code change.
Search	Keyword and file-pattern search, plus a built-in hybrid index for semantic search	A managed search service, chosen once it is known where and how CSA data is stored (Azure AI Search is one option), behind the same search tools
Tools & sessions	One Docker container per conversation	Azure Container Apps sessions
Activity record	pi's append-only session log	The same log, forwarded to Azure immutable storage
Sign-in	None (internal prototype)	CSA identity provider with CSA roles

6. Technology stack and why

The heart of CRAFT is the agent loop (Figure 3). The model reads the question and decides on a next step. If that step is a tool, the request passes the checkpoint, the tool runs in the conversation's Docker container, and the result comes back for the model to weigh. The loop repeats until the model has enough evidence to answer, and the answer's citations are checked before anyone sees them. Every pass through the loop is written to the activity record. The stack is chosen to make that loop capable, controllable and traceable.

Layer	Choice	Why
Agent framework	pi (TypeScript)	Runs the core loop. Built for autonomous agents that plan, use tools and iterate, rather than following a fixed script. Every tool use can be checked before it runs and inspected after, which is the control point the requirements specify (UMR-012). It reports tokens and cost for every step, and it writes an append-only log of every message, tool call and result that can be used to branch or resume a conversation. It works with Azure AI Foundry and dozens of other model providers.
Language model	OpenAI GPT-5 on Azure AI Foundry	Strong reasoning over long technical documents. The model is set by configuration, so it can be replaced without code changes (UMR-081).
Tools & sessions	Docker, one container per conversation	Every tool the agent uses runs inside the conversation's own container. It has read-only system files, no administrator rights, no network access, and fixed CPU, memory and process limits. Documents are mounted read-only, and the container is removed when the conversation ends (UMR-092, UMR-093).
Agent runtime	Node.js with Express	A small, standard web server. It hosts one agent per conversation, manages each conversation's container, checks citations, and streams results to the browser.
Search	Keyword and file-pattern search today. Semantic search next, with Orama (hybrid index) and OpenAI text-embedding-3 on Azure AI Foundry	The agent picks the right method for each question: exact terms for a requirement ID, meaning-based search for a concept. Orama is an off-the-shelf TypeScript search engine that runs inside the agent runtime. It combines keyword (BM25) and vector results in one ranking with adjustable weights, matching the 0.7 semantic / 0.3 keyword baseline (UMR-004, UMR-005). Documents are split along the same headings, pages and rows used for citations, so every semantic hit comes back ready to cite. A production search service can replace it later behind the same tools.
Document preparation	mammoth (Word), pdf.js (PDF), ExcelJS (Excel)	Converts each approved document once into searchable text with location markers, plus a fingerprint of its exact contents. That fingerprint is what makes citations verifiable.
Language	TypeScript throughout	One language from the browser to the agent. Shared definitions mean fewer mismatches between parts, and one skill set to maintain.

User interface	React with assistant-ui	Built for AI chat. Answers stream in as they are written, each search step is shown, and source cards and approve/deny cards come built in. Clicking a citation opens the original document at the cited spot. It runs entirely on CRAFT's own servers and calls no outside services.
-----------------------	-------------------------	---

7. Key design decisions

Check citations, don't trust them

Language models can invent plausible references. CRAFT treats every citation as a claim to verify. It must name a real document, its exact version (a fingerprint of its contents) and a location that exists in that document. Anything that fails the check is never shown as a source (UMR-006).

Prepare documents once, up front

Converting documents ahead of time gives the agent clean, searchable text and gives every passage a stable address. The agent works on these prepared copies, read-only. The original files are never placed in its reach (UMR-051).

One sealed Docker container per conversation

Each conversation's agent, workspace and tools are created together and removed together. Idle conversations close after an hour, and anything left over is cleaned up automatically when the system restarts (UMR-092, UMR-095). The activity record is separate, so what the agent did can be kept after its workspace is gone.

Keep user identity away from the agent

The planned app backend owns users, roles and approvals. The agent runtime only runs agents. This keeps the agent's reach small and makes separation of duties straightforward: the person who asks can never approve their own request (UMR-065.1).

No hidden outside services

Neither the user interface nor the agent framework sends data anywhere in this configuration. The only outside call is to the language model, which will run in a Canadian Azure region.

8. How the requirements are addressed

Working built and running ·
 Partial partly in place ·
 Next next build stage ·
 Planned production phase

Requirement	How CRAFT meets it	Status
Conversations with one accountable agent UMR-001, 011, 095	Each conversation gets its own agent, which lasts for the whole conversation. The agent, its working context and its workspace end together.	Working
Private, temporary workspace UMR-092	A dedicated folder and Docker container per conversation, removed when it ends. Letting users list and download their files comes next.	Working
Every fact cited with document, version and location UMR-002, 006	Each citation carries a document ID, a content fingerprint and a heading, page or row. Any citation that doesn't match the catalog is rejected.	Working
Read only from approved sources UMR-047, 051	The agent sees only prepared copies of the approved collection, read-only.	Working
Selective evidence UMR-003	The agent pulls in only the passages that match its search, not the whole collection.	Working
Replaceable language model UMR-081	The model is chosen in configuration.	Working
Agent chooses how to search UMR-094, 009	The agent picks its own search terms and methods and searches again when the first results aren't enough. Keyword and file-pattern search work today, through tools with named inputs (folder, file pattern, text pattern). Semantic search (Orama with Foundry embeddings) is the remaining piece.	Partial
Documents are data, not instructions UMR-068	The agent is instructed to ignore instructions found in documents. The checkpoint will add labelling and detection, measured against a test set.	Partial
Say so when evidence is missing UMR-086	The agent is instructed to decline or ask rather than guess. This will be tested during acceptance testing.	Partial
Contained environment for analysis code UMR-093	A Docker container per conversation with read-only system files, no administrator rights, no network access and fixed resource limits. It is removed when the conversation ends.	Working
Complete activity record UMR-027, ARR-010	pi writes every message, tool call (with its inputs) and result to an append-only session log, which records the search path behind each answer. The same log lets a conversation be branched or resumed. Forwarding it to immutable Azure storage is part of the production setup.	Working
Approved tool list, budgets, loop breaker UMR-012, 013, 019, 030	The checkpoint on every tool call (Figure 3). The framework already reports token use and cost for each step.	Next
Human approval of high-risk actions UMR-021–026	Pause, save, review and resume (Figure 5).	Next

Classification and simulated-data labels UMR-058, 067, 091	Labels recorded at document preparation. A "simulated" badge on every source card that comes from simulated data.	Next
Keyword + semantic search, combined UMR-004, 005	A hybrid Orama index with OpenAI text-embedding-3 embeddings from Azure AI Foundry. Keyword and semantic results are merged into one weighted ranking (0.7 / 0.3), with duplicates removed. In production this moves to a managed search service.	Next
Confidence score UMR-007, 008	Computed from evidence coverage, citation checks and agreement between sources. Low scores go to a reviewer.	Planned
Sign-in, roles, separation of duties UMR-065	Sign-in through CSA's approved identity provider, handled by the app backend, with the roles defined in the requirements.	Planned
Hosted in Canada, Protected B UMR-058, 070	Azure Canadian region inside a Protected B landing zone (Figure 6).	Planned
Acceptance data set UMR-064	Questions with known answers and source passages, used to measure search quality, grounding and confidence.	Planned

9. Roadmap

Working now

- Autonomous agent loop that picks and repeats its own searches
- One agent and one Docker container per conversation
- Append-only activity record of every step
- Verified citations with one-click previews
- Document preparation with fingerprints and locations
- Chat with answers streamed live

Next

- Semantic search: Orama hybrid index + Foundry embeddings
- Checkpoint: approved tools, budgets, loop breaker
- Approval pause and resume
- Classification and simulated-data labels
- Acceptance data set

Production

- Managed search service, chosen to fit how CSA data is stored
- CSA sign-in and roles
- Azure Container Apps sessions
- Immutable Azure storage for the record
- Azure AI Foundry in a Canadian region
- Confidence scores and reviewer routing

10. Proposal: let the agent use a shell inside its container

The requirements currently rule out free-form shell access for agents (§1.3, UMR-094, ASG-012). This section proposes a change request to allow it inside each conversation's own Docker container, starting in Phase 1. The named search tools and semantic search would stay in place alongside it.

Why the shell is worth having

Agents are exceptionally good at it. Today's models have learned from enormous amounts of command-line work. They combine standard tools like `rg`, `find`, `sed` and `awk` to search, filter and cross-check in a single step: find every mention of a mission, narrow it to cost tables, then count the rows. A set of named tools can only offer the combinations someone thought to build in advance.

The industry has moved this way. The team behind Anthropic's Claude Code reported that early versions used a vector database, but they "found pretty quickly that agentic search generally works better. It is also simpler and doesn't have the same issues around security, privacy, staleness, and reliability." Cursor reached a complementary result: its agent "makes heavy use of `grep`," and adding semantic search on top of it improved accuracy by 12.5% on average. The best results come from having both.

It already covers the keyword search CRAFT needs. Exact-term search, file-pattern search and reading a passage by location are all things the shell does natively. The named tools reproduce a subset of that capability. Semantic search adds what the shell can't do, so the two complement each other, which fits the nature of CSA's documents.

The requirement's concerns, and how the container answers them

The requirements document explains why shells were excluded: the command space is "unbounded, unauditable, and open to injection from ingested content" (Annex F, RC-17). Each concern is addressed by the design already in place.

Concern	How it is answered
Unbounded	The commands are many, but their reach is small. The container has no network access, read-only system files, no administrator rights, and fixed CPU, memory and process limits. Documents are mounted read-only. The container can't see the host, the internet or any other conversation, and it is removed when the conversation ends.
Unauditable	Every command, its output and its result are written to the activity record and shown in the interface. A reviewer sees the exact command that was run, which is often clearer than reading a tool's parameters.
Open to injection	A hidden instruction in a document can only act inside a sealed, offline container that holds read-only copies of documents the user can already see. There is nowhere to send data and nothing outside to change. The checkpoint can also inspect every command before it runs, and block or flag patterns.

The requirements already allow agent-generated analysis code to run in this same kind of containment (UMR-093, ASG-011). A shell in the same container adds little new exposure.

Proposed conditions

- Shell access only inside the conversation's own container, with network access off and documents read-only.
- Every command recorded in the activity record and visible to the user.
- The checkpoint inspects commands before they run and can deny them.
- Hybrid keyword and semantic search remains the default search path (UMR-004).
- After Phase 1, compare answer quality with and without the shell on the acceptance data set (UMR-064), and decide whether to keep it.

Requested decision: submit a change through CSA's document control process (§1.4) to amend §1.3, UMR-094 and ASG-012, so that shell access is permitted inside the contained session environment under the conditions above.

Sources. Boris Cherny (Claude Code, Anthropic), x.com/bcherny/status/2017824286489383315 · Cursor, "Improving agent with semantic search," cursor.com/blog/semsearch